

COMPUTER GRAPHICS

Unit-II: Output Primitives

Comprehensive Study Notes

Introduction to Output Primitives

Definition

Output primitives are the basic geometric entities used to construct more complex graphical objects. They form the building blocks of computer graphics and include points, lines, circles, ellipses, curves, polygons, and characters.

Classification

Geometric Primitives	Curves	Text/Character Primitives
Point, Line, Polygon, Rectangle, Triangle	Circle, Ellipse, Arc, Splines, Bezier Curves	Bitmap Fonts, Outline Fonts, Stroke Fonts, Character Generation

1. Line Drawing Algorithms

Introduction

A line is the most basic output primitive. In computer graphics, lines are drawn on a raster display by illuminating pixels that approximate the ideal line path between two endpoints.

Line Specification

- Defined by two endpoints: (x_1, y_1) and (x_2, y_2)
- Equation: $y = mx + b$
- Slope: $m = (y_2 - y_1) / (x_2 - x_1)$
- Intercept: $b = y_1 - (m \times x_1)$

Properties of Good Line Drawing Algorithm

- **Accurate** – Close to ideal line
- **Straight** – No jaggies/artifacts
- **Efficient** – Fast computation
- **Uniform** – Constant brightness along line
- **Endpoints** exactly at specified points
- Works for all slopes

1.1 DDA (Digital Differential Analyzer) Algorithm

DDA is a line drawing algorithm that computes intermediate points of a line using floating-point arithmetic and rounding.

Working Principle

- Uses floating-point arithmetic
- Increments x and y by small steps
- Step size determined by slope
- Rounds coordinates to nearest integer

Algorithm Steps

```

1. Input: (x1, y1) and (x2, y2)
2. dx = x2 - x1, dy = y2 - y1
3. if |dx| > |dy|: steps = |dx| else steps = |dy|
4. xinc = dx / steps, yinc = dy / steps
5. x = x1, y = y1
6. for i = 0 to steps:
    plot(Round(x), Round(y))
    x = x + xinc, y = y + yinc

```

Step-by-Step Example

Given: Line from (2,3) to (8,6)

Step	Calculation
1	$dx = 8 - 2 = 6$, $dy = 6 - 3 = 3$
2	$ dx = 6$, $ dy = 3$. $ dx > dy $, so steps = $ dx = 6$
3	$xinc = 6/6 = 1.0$, $yinc = 3/6 = 0.5$

i	x	y	Round(x)	Round(y)
0	2.0	3.0	2	3
1	3.0	3.5	3	4
2	4.0	4.0	4	4
3	5.0	4.5	5	5
4	6.0	5.0	6	5
5	7.0	5.5	7	6
6	8.0	6.0	8	6

Final Pixels: (2,3), (3,4), (4,4), (5,5), (6,5), (7,6), (8,6)

Advantages & Disadvantages

Advantages	Disadvantages
Simple to understand and implement	Uses floating-point arithmetic (slower)

Works for all slopes (0 to infinity)	Rounding errors accumulate
No special cases needed for different slopes	Not suitable for high-performance graphics
Uses fixed number of iterations	More time-consuming than Bresenham's
Good for drawing lines on digital computers	Requires round() function in each iteration

1.2 Bresenham's Line Drawing Algorithm

Bresenham's Line Drawing Algorithm is an efficient, incremental line drawing algorithm that uses only integer arithmetic to determine the points of a line, eliminating the need for floating-point operations.

Working Principle

- Uses only integer arithmetic
- Incremental decision parameter
- Decides between two possible pixel positions
- Very efficient

Algorithm Steps (Slope < 1, $|m| \leq 1$)

```
1. Input: (x1, y1) and (x2, y2)
2. dx = x2 - x1, dy = y2 - y1
3. p = 2dy - dx
4. x = x1, y = y1
5. plot(x, y)
6. for i = 0 to dx:
    if p < 0:
        x = x + 1
        p = p + 2dy
    else:
        x = x + 1
        y = y + 1
        p = p + 2dy - 2dx
    plot(x, y)
```

Step-by-Step Example

Given: Line from (1,1) to (5,3)

Step	Calculation
1	$dx = 5 - 1 = 4$, $dy = 3 - 1 = 2$
2	$p = 2dy - dx = 2(2) - 4 = 0$
3	Plot initial point: (1,1)

i	x	y	p	Next p
0	1	1	0	$0 + 4 - 8 = -4$
1	2	2	-4	$-4 + 4 = 0$
2	3	2	0	$0 + 4 - 8 = -4$
3	4	3	-4	$-4 + 4 = 0$
4	5	3	0	END

Final Pixels: (1,1), (2,2), (3,2), (4,3), (5,3)

Advantages & Disadvantages

Advantages	Disadvantages
Uses only integer arithmetic	Complex to understand initially
Faster than DDA	Requires separate implementation for different slopes
No floating-point operations	Special cases for vertical and horizontal lines
No rounding errors	Cannot handle lines with slope > 1 without symmetry
Very efficient for hardware implementation	
Most widely used line drawing algorithm	

Bresenham's for All Slopes

Case	Description	Decision Parameter
Slope = 0 (Horizontal)	y constant, increment x only	—
Slope = ∞ (Vertical)	x constant, increment y only	—
$0 < m < 1$ (Gentle)	x increases by 1, y by 0 or 1	$p = 2dy - dx$
$ m > 1$ (Steep)	y increases by 1, x by 0 or 1	$p = 2dx - dy$
Negative Slope	Use absolute values, swap signs	—

1.3 DDA vs Bresenham's Comparison

Parameter	DDA	Bresenham's
Arithmetic	Floating-point	Integer arithmetic
Speed	Slower	Faster
Accuracy	Rounding errors accumulate	No rounding errors
Operations	Uses division	Uses only addition
Hardware	Complex hardware	Simple hardware
Understanding	Easy to understand	Slightly complex
Slopes	Works for all slopes	Needs different cases
Accuracy	Less accurate	More accurate
Time	More time consuming	Less time consuming

2. Circle Generating Algorithms

2.1 Midpoint Circle Drawing Algorithm

The Midpoint Circle Drawing Algorithm is an efficient integer-based algorithm that uses symmetry properties of circles to determine pixel positions on a circle path.

Circle Properties

- Equation: $x^2 + y^2 = r^2$
- Center: (x_c, y_c) , Radius: r
- **8-way symmetry:** $(x,y) \rightarrow (y,x) \rightarrow (y,-x) \rightarrow (x,-y) \rightarrow (-x,-y) \rightarrow (-y,-x) \rightarrow (-y,x) \rightarrow (-x,y)$

Algorithm Steps

```

1. Input: Center  $(x_c, y_c)$  and radius  $r$ 
2.  $x = 0, y = r$ 
3.  $p = 1 - r$ 
4. plot all symmetry points  $(x_c \pm x, y_c \pm y)$ 
5. while  $x < y$ :
     $x = x + 1$ 
    if  $p < 0$ :
         $p = p + 2x + 1$ 
    else:
         $y = y - 1$ 
         $p = p + 2x + 1 - 2y$ 
    plot all symmetry points
  
```

Step-by-Step Example

Given: Center $(0,0)$, radius $r = 10$

Initial: $x = 0, y = 10, p = 1 - 10 = -9$

Step	x	y	p	New p
0	0	10	-9	—
1	1	10	$-9 + 3 = -6$	-6

2	2	10	$-6 + 5 = -1$	-1
3	3	10	$-1 + 7 = 6$	6
4	4	9	$6 + 9 - 2 = 13$	13
5	5	8	$13 + 11 - 4 = 20$	20
6	6	7	$20 + 13 - 6 = 27$	27
7	7	6	STOP ($x \geq y$)	

Points generated (first octant): (0,10), (1,10), (2,10), (3,10), (4,9), (5,8), (6,7)

Complete circle using 8-way symmetry: (0,±10), (±10,0), (1,±10), (±10,1), (±1,±10), (±10,±1), (6,±7), (7,±6), (±6,±7), (±7,±6), ...

Advantages & Disadvantages

Advantages	Disadvantages
Uses only integer arithmetic	Only works for integer radius
Uses 8-way symmetry to reduce computation	Cannot handle non-integer centers directly
Efficient and fast	Not suitable for ellipses
No floating-point operations	Difficult to handle arbitrary circle parameters
No trigonometric functions needed	
Produces good circle approximation	

2.2 Bresenham's Circle Drawing Algorithm

Bresenham's Circle Drawing Algorithm is similar to the midpoint approach. It generates circle points using integer decision parameters by computing the closest pixel position to the actual circle path. The decision parameter determines whether the next pixel is plotted at the same y-coordinate or one step below. It also leverages 8-way symmetry, computing only one octant and reflecting points across the other seven octants.

3. Ellipse Generating Algorithm

3.1 Midpoint Ellipse Drawing Algorithm

The Midpoint Ellipse Drawing Algorithm is an extension of the midpoint circle algorithm that handles the asymmetry of ellipses using two decision parameters.

Ellipse Properties

- Equation: $x^2/rx^2 + y^2/ry^2 = 1$
- Center: (xc, yc)
- x-radius: rx, y-radius: ry
- **4-way symmetry**
- Divided into 2 regions (slope < 1 and slope > 1)

Algorithm Steps

```

1. Input: Center (xc, yc), rx, ry
2. x = 0, y = ry
3. p1 = ry2 - rx2·ry + rx2/4
4. plot symmetry points (xc±x, yc±y)
5. Region 1 (slope > -1):
   while 2·ry2·x < 2·rx2·y:
       x = x + 1
       if p1 < 0: p1 = p1 + 2·ry2·x + ry2
       else: y = y - 1, p1 = p1 + 2·ry2·x - 2·rx2·y + ry2
6. Region 2 (slope < -1):
   p2 = ry2·(x+½)2 + rx2·(y-1)2 - rx2·ry2
   while y > 0:
       y = y - 1
       if p2 > 0: p2 = p2 - 2·rx2·y + rx2
       else: x = x + 1, p2 = p2 + 2·ry2·x - 2·rx2·y + rx2

```

Circle vs Ellipse Algorithm Comparison

Parameter	Circle	Ellipse
Symmetry	8-way symmetry	4-way symmetry
Parameters	One parameter (r)	Two parameters (rx, ry)

Decision params	One decision parameter	Two decision parameters (p1, p2)
Regions	One region	Two regions
Complexity	Simpler	More complex
Speed	Faster	Slower
Initial p	$p = 1 - r$	p1 and p2 (separate formulas)

4. Polygon

4.1 Polygon Types

- **Convex Polygon** – All interior angles $< 180^\circ$. Every line between any two points lies inside the polygon.
- **Concave Polygon** – At least one interior angle $> 180^\circ$. Some lines between points exit the polygon.
- **Complex (Self-intersecting)** – Edges cross each other.

4.2 Polygon Filling Algorithms

Scan-Line Fill Algorithm

- Processes the polygon one scan line at a time (horizontally).
- Finds intersection points of the scan line with polygon edges.
- Sorts intersection points by x-coordinate.
- Fills pixel pairs between sorted intersections (odd-even parity rule).
- Handles special cases: vertices, horizontal edges.
- Efficient for convex and simple concave polygons.

Flood Fill Algorithm

- Starts from a seed point inside the polygon.
- Recursively or iteratively fills neighboring pixels (4-connected or 8-connected).
- **4-connected**: Fills up, down, left, right neighbors.
- **8-connected**: Fills all 8 surrounding neighbors (diagonals included).
- Used for filling arbitrarily shaped regions.
- **Boundary Fill**: Fills until a boundary color is encountered.

Boundary Fill Algorithm

- Requires a seed point inside the region to be filled.
- Checks neighboring pixels recursively.
- If neighbor is not the boundary color and not already filled, set it to fill color.
- Continue until the entire region is filled.

- Can be implemented with a stack to avoid deep recursion.

Algorithm	Approach	Best For
Scan-Line Fill	Horizontal scanning, intersection computation	Convex/concave polygons, efficiency
Flood Fill	Seed-based recursive expansion	Arbitrary shapes, interactive tools
Boundary Fill	Seed-based with boundary detection	Simple regions with well-defined boundaries

5. Anti-aliasing

Definition

Anti-aliasing is a technique used in computer graphics to reduce or eliminate aliasing artifacts (jaggies, stair-step effects) that appear on raster displays when rendering smooth curves, lines, or diagonal edges.

Causes of Aliasing

- Insufficient sampling resolution in raster displays.
- Sudden transitions between pixel colors (staircase effect).
- Undersampling of continuous signals (Nyquist theorem violation).

Anti-aliasing Techniques

Technique	Description
Supersampling (SSAA)	Renders at higher resolution and downsamples. Each pixel is sampled at multiple sub-pixel positions and averaged.
Multisampling (MSAA)	Similar to supersampling but shares samples across adjacent pixels. More efficient.
Area Sampling	Computes the area of the pixel covered by the primitive and sets intensity proportionally.
Pixel Phasing	Micro-positioning of pixels to better align with the ideal line.
Filtering Techniques	Applies low-pass filters (box, Gaussian, sinc) to smooth edges.

Supersampling Example

A pixel is divided into $4 \times 4 = 16$ sub-pixels. The color/intensity of the pixel is the average of all sub-pixel samples that lie on or near the line. This produces smoother edges at the cost of increased computation.

Benefits of Anti-aliasing

- Produces smoother curves and diagonal lines.
- Reduces visual artifacts (jaggies, moiré patterns).
- Improves overall image quality.
- Essential for high-quality graphics rendering.

6. Character Generation

Methods of Character Generation

Method	Description	Examples
Bitmap (Raster) Fonts	Stored as pixel matrix. Pre-rendered characters. Fast display. Quality depends on resolution. Not scalable without quality loss.	5x7, 8x8 matrices
Outline (Vector) Fonts	Stored as mathematical outlines (Bezier curves). Scalable to any size. High quality at any resolution. More processing.	TrueType, OpenType
Stroke Fonts	Stored as line segments. Simple and fast. Used in vector displays. Limited quality.	Hershey fonts

Bitmap Font Example – Character 'A' (5×7 Matrix)

```
Row 0: 0 1 1 1 0
Row 1: 1 0 0 0 1
Row 2: 1 0 0 0 1
Row 3: 1 1 1 1 1
Row 4: 1 0 0 0 1
Row 5: 1 0 0 0 1
Row 6: 0 0 0 0 0
```

Character Generation Methods Comparison

Feature	Bitmap Fonts	Outline Fonts	Stroke Fonts
Storage	Large	Medium	Small
Scalable	No	Yes	Yes
Quality	Fixed	Excellent	Poor
Speed	Fast	Medium	Fast
Smooth	No	Yes	No (lines only)

Memory	High	Medium	Low
Use	CRT, LCD	Print, Web	CAD, Plotters

Quick Reference Summary – Unit II

Key Terms

1. **Output Primitive** – Basic geometric entity for graphics
2. **DDA** – Digital Differential Analyzer (floating-point line algorithm)
3. **Bresenham's** – Integer-only line algorithm
4. **Midpoint Algorithm** – Decision parameter-based algorithm
5. **Circle Symmetry** – 8-way symmetry for circle drawing
6. **Ellipse Symmetry** – 4-way symmetry for ellipse drawing
7. **Bitmap Font** – Pixel-based character storage
8. **Outline Font** – Mathematical outline-based character storage
9. **Stroke Font** – Line segment-based character storage
10. **Anti-aliasing** – Technique to reduce jaggies

Algorithm Complexity

Algorithm	Complexity
DDA	$O(n)$ – n = number of steps
Bresenham's Line	$O(n)$ – n = number of x increments
Midpoint Circle	$O(r)$ – r = radius
Midpoint Ellipse	$O(r_x + r_y)$ – r_x, r_y = radii

Important Formulas

Entity	Formula
Line	$y = mx + b$, $m = (y_2 - y_1)/(x_2 - x_1)$
Circle	$x^2 + y^2 = r^2$
Ellipse	$x^2/r_x^2 + y^2/r_y^2 = 1$
DDA Step	$\text{steps} = \max(dx , dy)$

Bresenham p	$p = 2dy - dx$
Circle p	$p = 1 - r$

When to Use

Situation	Recommended
Floating-point available, simple implementation	DDA
Speed important, integer-only hardware	Bresenham's Line
Efficient circle drawing	Midpoint Circle
Efficient ellipse drawing	Midpoint Ellipse
Small, fixed-size text display	Bitmap Fonts
Scalable, high-quality text	Outline Fonts
Vector displays, CAD applications	Stroke Fonts

Questions

- Q1.** Define output primitives. Explain the classification of output primitives with examples.
- Q2.** Explain DDA line drawing algorithm with a suitable example. List its advantages and disadvantages.
- Q3.** Explain Bresenham's line drawing algorithm with a step-by-step example. How is it better than DDA?
- Q4.** Compare DDA and Bresenham's line drawing algorithms in detail.
- Q5.** Explain the Midpoint Circle Drawing Algorithm with an example ($r = 10$).
- Q6.** Explain Bresenham's circle drawing algorithm. How does it use 8-way symmetry?
- Q7.** Explain the Midpoint Ellipse Drawing Algorithm. How is it different from the midpoint circle algorithm?
- Q8.** Compare circle and ellipse generating algorithms.
- Q9.** Explain polygon filling algorithms: scan-line fill, flood fill, and boundary fill.
- Q10.** Differentiate between convex and concave polygons.
- Q11.** What is anti-aliasing? Explain various anti-aliasing techniques used in computer graphics.

Q12. Explain the different methods of character generation (bitmap, outline, stroke fonts).

— End of Computer Graphics — Unit II Notes —